

Software Development Fundamentals (SDF) – I

15B11CI111

ODD 2023-24



Lecture 4

Outline of Lecture

- 1. C Programming – Introduction and General Structure**
- 2. C Compiler – Online and Offline**
- 3. Code Blocks – Download and Installation**
- 4. Code Blocks – Brief User Manual**
- 5. Arithmetical Operations – Flow Chart and C Program**
- 6. Exercises**

C Programming – Introduction and General Structure

1. C is a structured programming language
2. C program consists functions (user defined functions and library functions) and variables
3. Functions contain instructions corresponding to the computations and operations to be performed, whereas variables stores values and used during computations
4. `main()` in c programming is user defined as well as predefined. Because **`main()` is programmer/user implemented function, whose prototype is predefined in the compiler.**
5. `main()` function is the entry and exit point of a C program & must be there in a C program

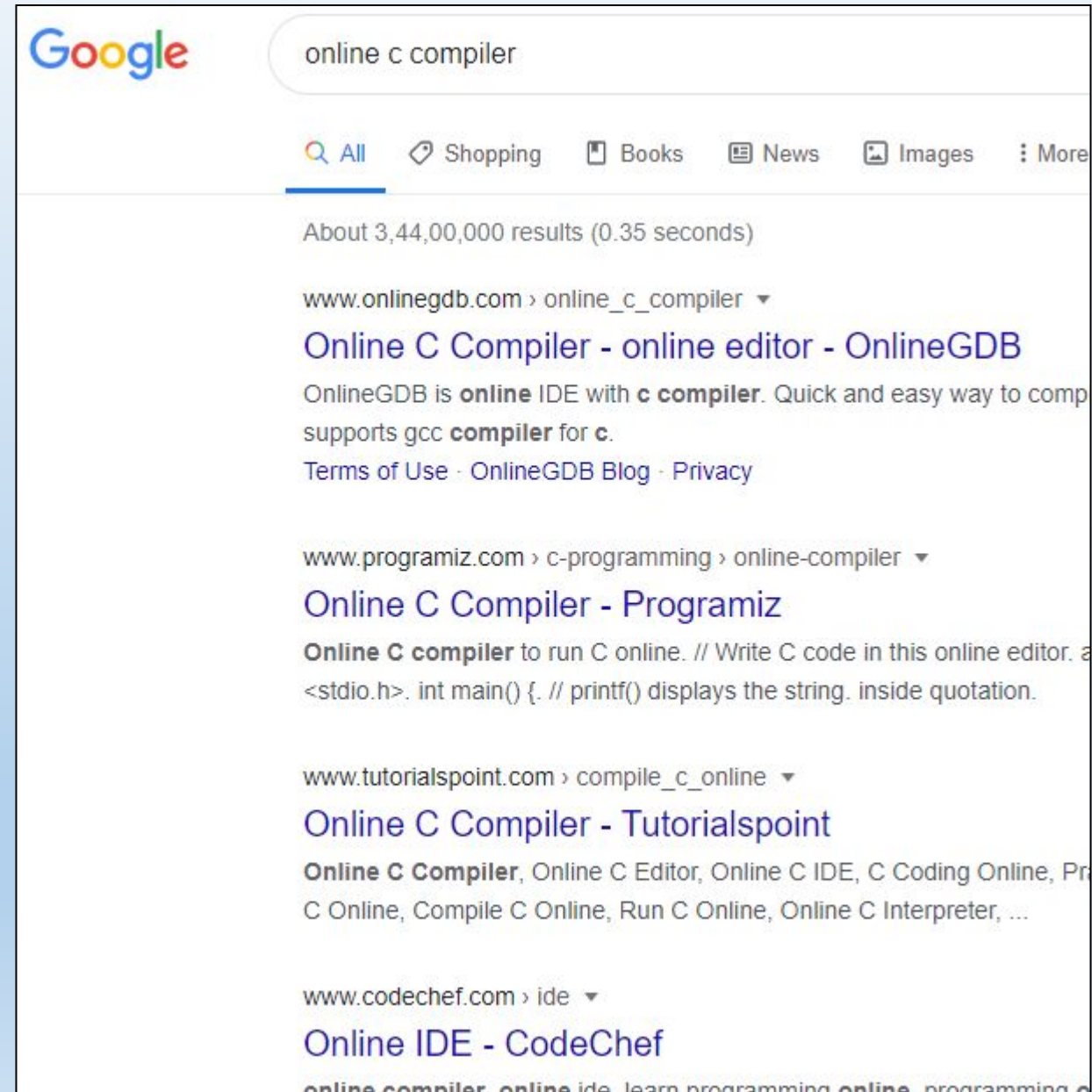
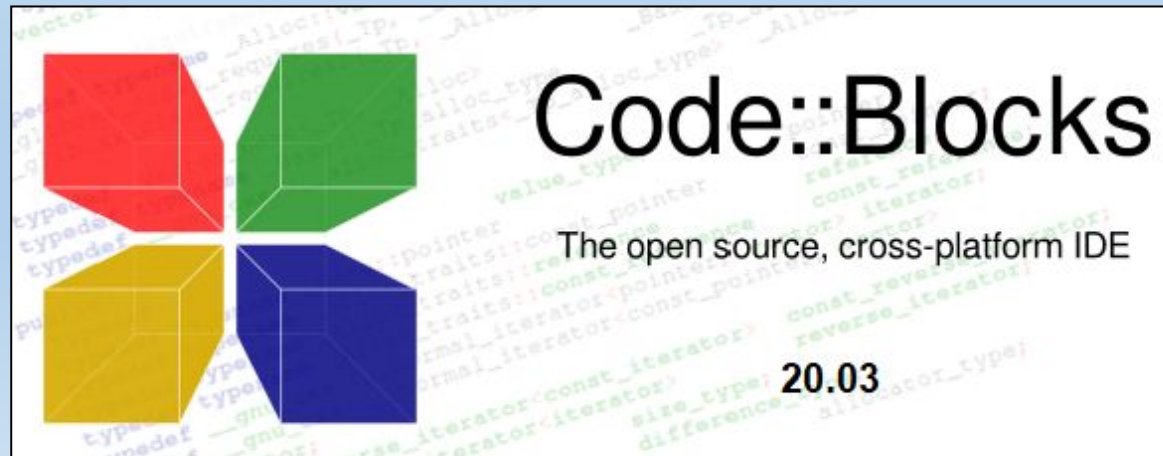
General Structure

1. Include Header Files
2. main function/method
3. Variable declaration
4. Body of the function/method
5. Return

```
#include <stdio.h>
int main()
{
    int A, B;
    printf("Enter an integer value");
    scanf("%d", &A);
    B = A * 5;
    printf("%d X 5 is %d",A, B);
    return 0;
}
```

C Compiler – Online and Offline

1. C is a high level language
2. Instructions are user's friendly
3. Not understood by Computers
4. Need to convert into machine understandable language, *i.e.* Machine Language
5. C compiler converts the program into M/c Language, *i.e.* we need to compile the program for its execution
6. gcc is one of the C compilers
7. Many online C compilers are available to compile your program
8. Code Blocks is one of the platforms (IDE) which facilitate you to compile the program offline



Code Blocks – Download and Installation



Code::Blocks

Code::Blocks - The IDE with all the features you need, having a consistent look, feel and operation across platform

- Home
- Features
- Downloads
- Forums
- Wiki

Main

- Home
- Features
- Screenshots
- Downloads
 - Binaries
 - Source
 - SVN
- Plugins
- User manual
- Licensing
- Donations

Quick links

- FAQ
- Wiki
- Forums
- Forums (mobile)
- Nightlies
- Ticket System
- Browse SVN
- Browse SVN log

Please select a setup package depending on your platform:

- Windows XP / Vista / 7 / 8.x / 10
- Linux 32 and 64-bit
- Mac OS X

NOTE: For older OS'es use older releases. There are releases for many OS version and platforms on the [Sourceforge.net](#) page.

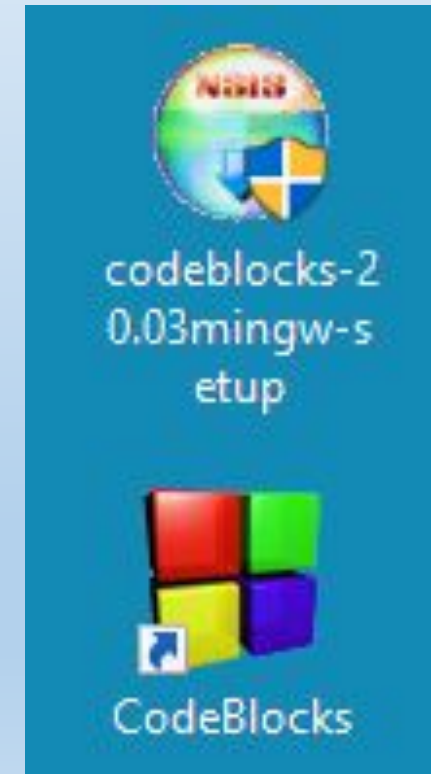
NOTE: There are also more recent *nightly builds* available in the [forums](#) or (for Ubuntu users) in the [Ubuntu PPA repository](#). Please note that we consider nightly builds to be *stable*, usually.

NOTE: We have a [Changelog](#) for 20.03, that gives you an overview over the enhancements and fixes we have put in the new release.

NOTE: The default builds are 64 bit (starting with release 20.03). We also provide 32bit builds for convenience.

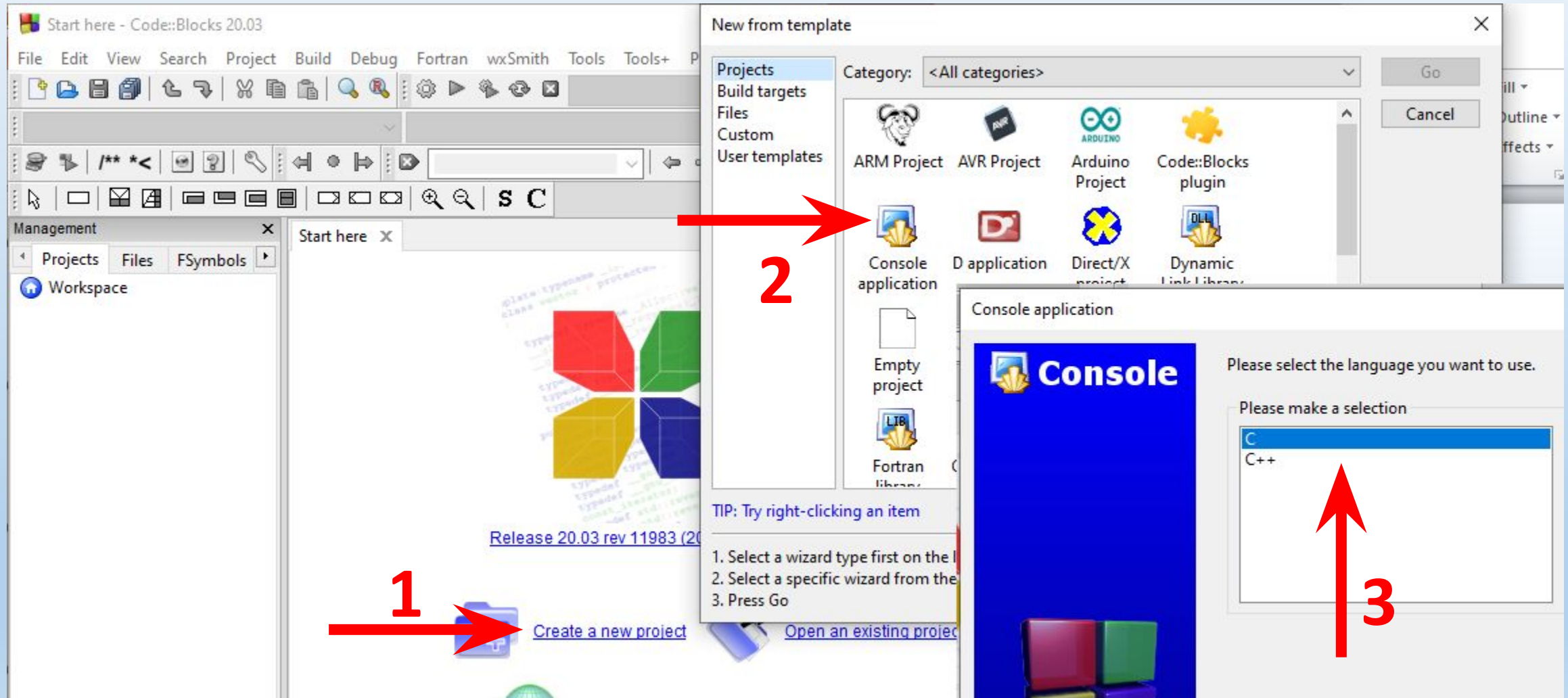
**Windows XP / Vista / 7 / 8.x / 10:**

File	Date	Download from
codeblocks-20.03-setup.exe	29 Mar 2020	FossHUB or Sourceforge.net
codeblocks-20.03-setup-nonadmin.exe	29 Mar 2020	FossHUB or Sourceforge.net
codeblocks-20.03-nosetup.zip	29 Mar 2020	FossHUB or Sourceforge.net
codeblocks-20.03mingw-setup.exe	29 Mar 2020	FossHUB or Sourceforge.net
codeblocks-20.03mingw-nosetup.zip	29 Mar 2020	FossHUB or Sourceforge.net
codeblocks-20.03-32bit-setup.exe	02 Apr 2020	FossHUB or Sourceforge.net

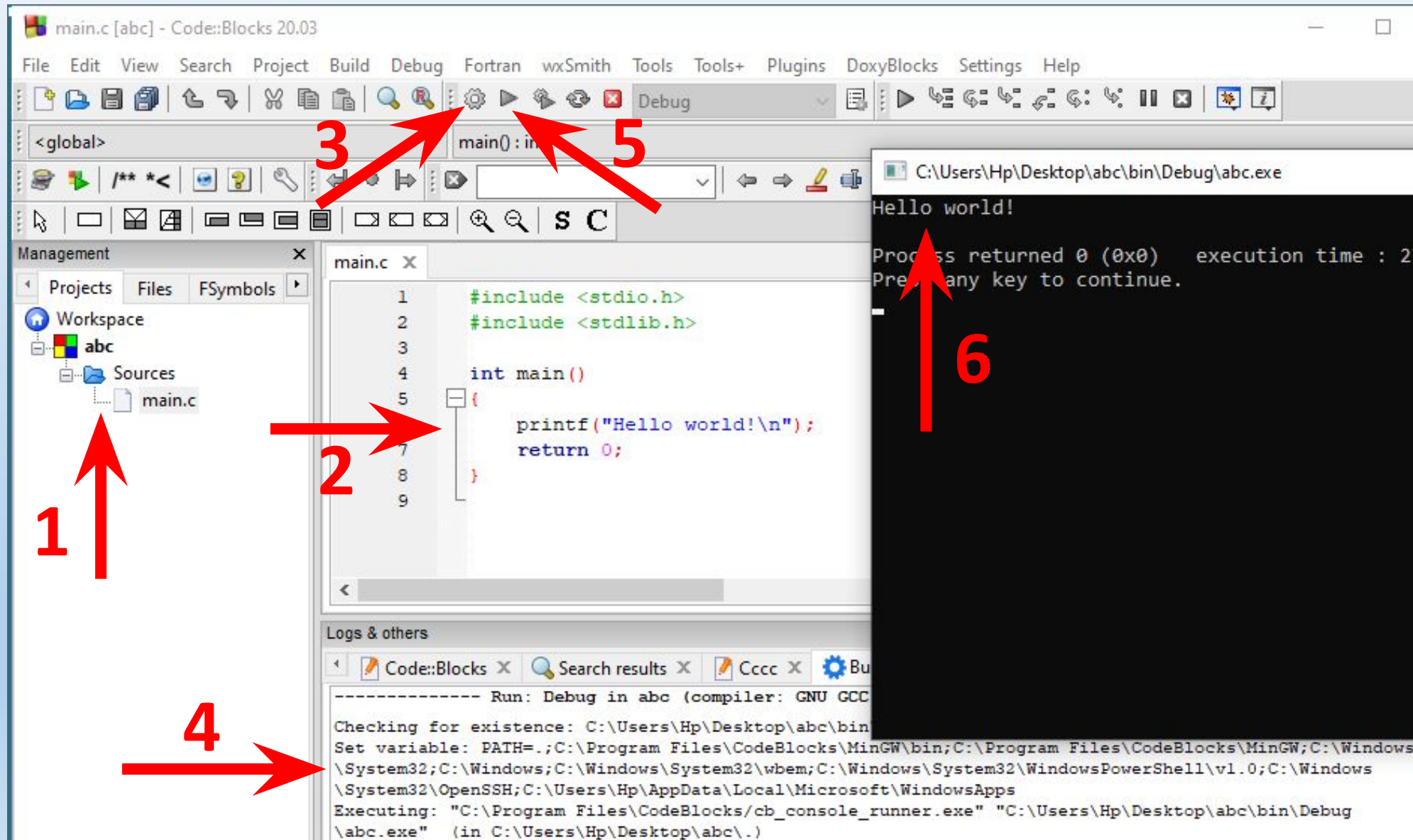


<http://www.codeblocks.org/downloads/26>

Code Blocks – Brief User Manual



Code Blocks – Brief User Manual

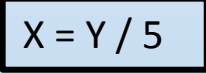
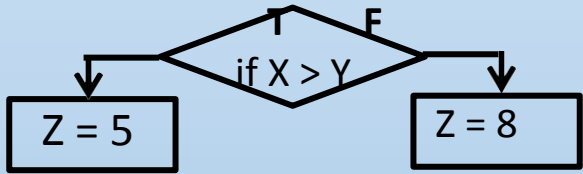
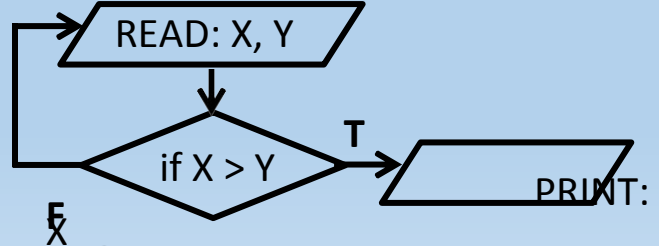
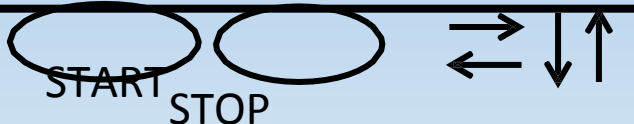


1. Source File, main
2. Code Block editor having C program
3. Compile / Build
4. Compilation / Run message display block
5. Run / Execute
6. Output popup window

Representation of Logic/Algorithm

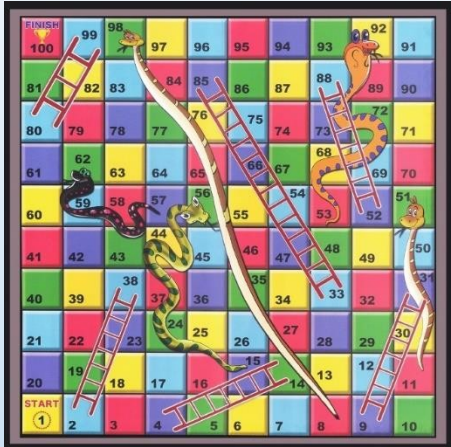
- | | | |
|-------------------------------------|---|---|
| 1. Representation using Steps | □ | Step by step representation of the Logic |
| 2. Representation using Flow Chart | □ | Pictorial (defined symbols) representation of the Logic |
| 3. Representation using Pseudo Code | □ | Represented in form of pseudo coding |

Common operations to be

Operations	Purpose with example	Flow chart symbols/pictures	Pseudo code
Input/ Output	Read the count, Print the count		READ: count PRINT: count
Arithmetical Operations & Process	Perform arithmetical operations, $X = Y / 5$		$X = Y / 5$
Conditional Operations	Check some conditions for true/false and perform operations accordingly if $X > Y$ then $Z = 5$ otherwise $Z = 8$		if $X > Y$, then $Z = 5$ else $Z = 8$
Repetition Operations	Repeat some operations until the required condition is met Read X & Y until $X > Y$. If $X > Y$, then do not repeat and display X.		READ: X, Y if $X < Y$, repeat READ: X, Y PRINT: X
Start / Stop / Flow	Beginning & End of the Logic Flow: direction of flow in the Logic		begin operation 1; operation 2;

Draw Flow Chart for Snake & Ladder Game

An incomplete flow chart is presented. **Complete it & discuss your solution in Tutorial**



Source: <http://www.nepaliclass.com/wp-content/uploads/2018/04/snake-and-ladder-board-game.jpg>

Two Players: P1 & P2

Two Tokens: T1 & T2

Dice: D (between 1 & 6)

Tokens Initial

Position:

T1 = 0 & T2 = 0

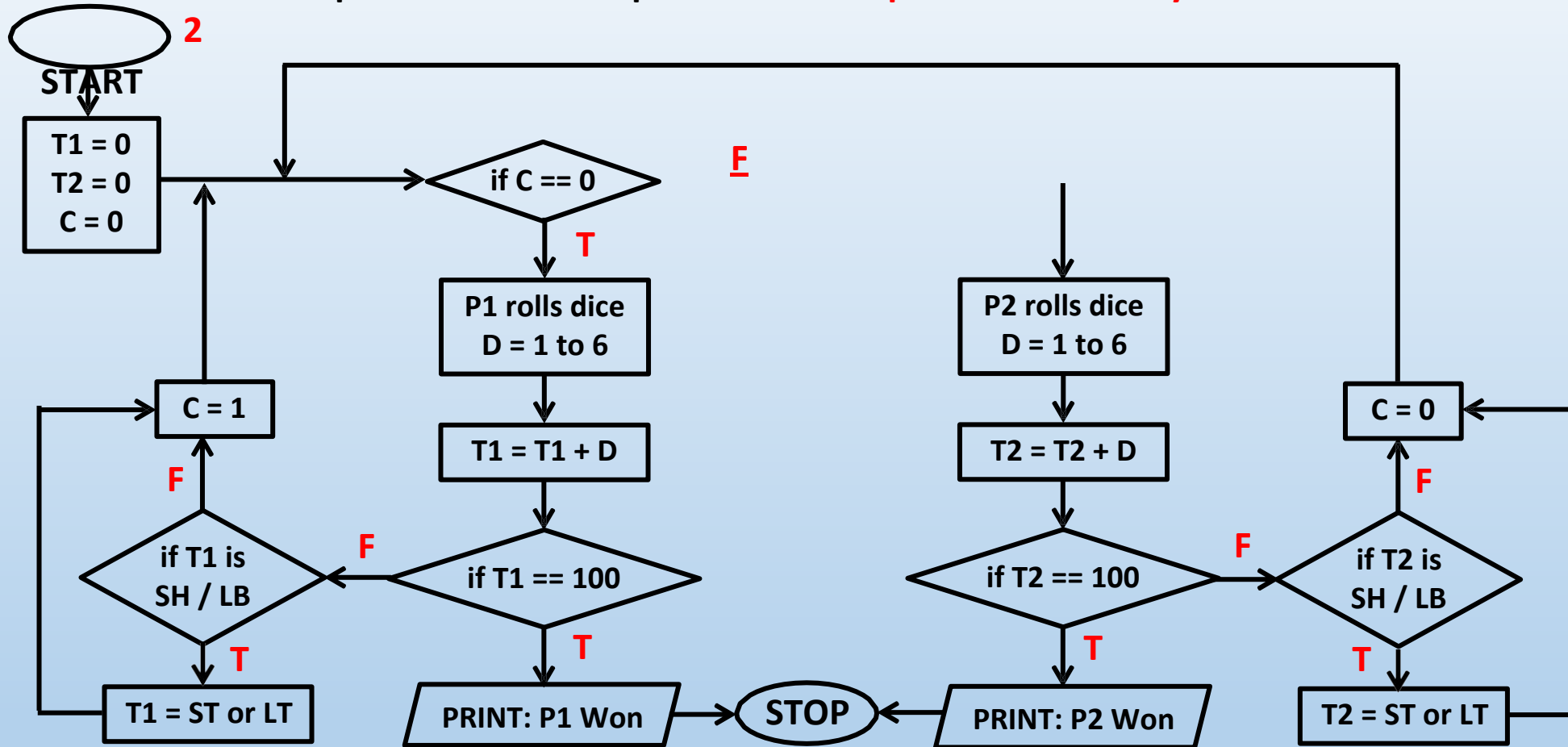
Turn / Chance: C

C = 0 (P1), C = 1 (P2)

Winning Condition:

either T1 lands at 100

T2 lands at 100



Q1. Did we miss something (requirements/logic) to be included in the flow chart?

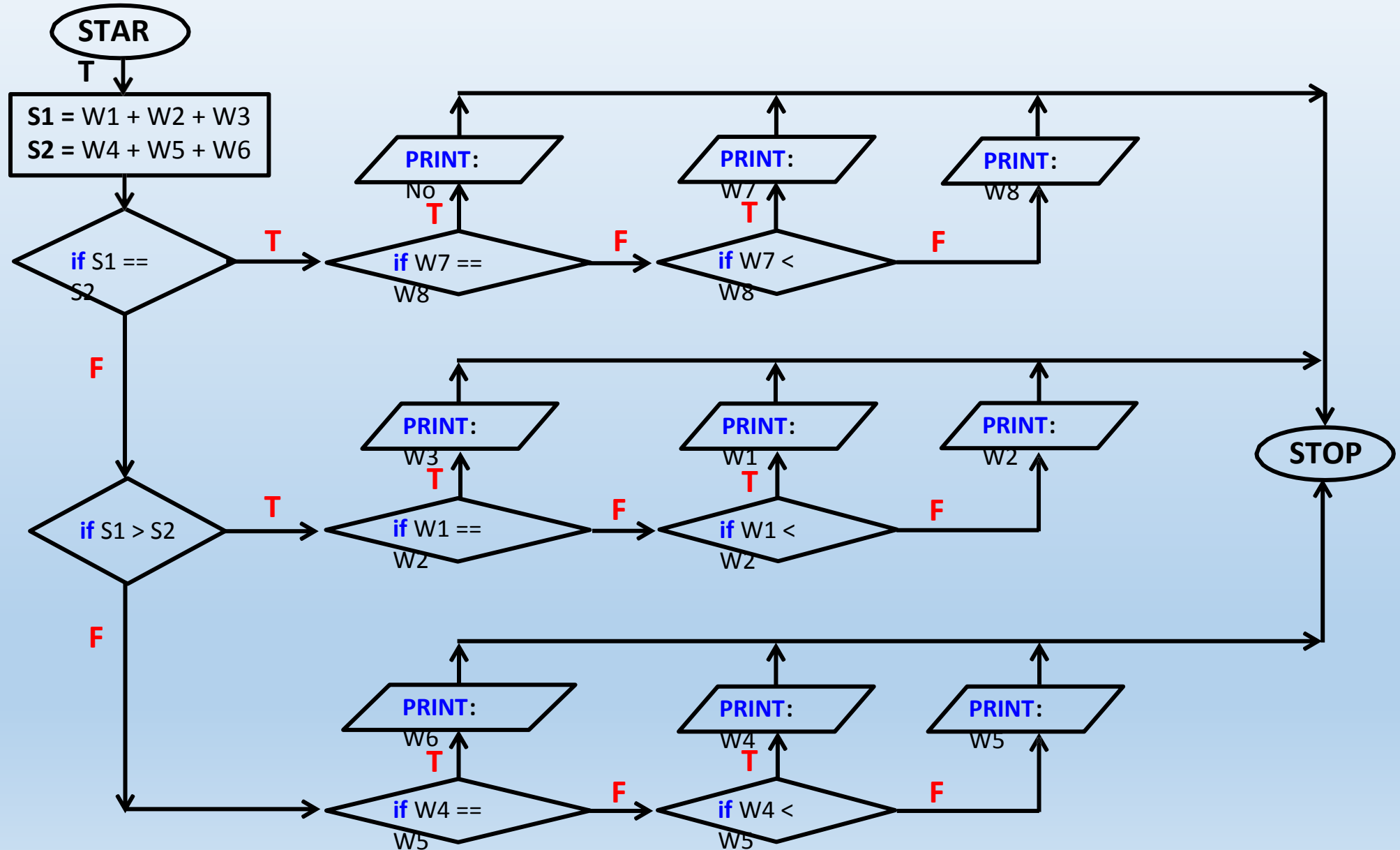
Ans. Yes. **(1)** If D is 6 then re rolling of dice is required. **(2)** If $T1 = (T1 + D)$ or $T2 = (T2 + D)$ is > 100

Q2. Anything else we have missed to include in the flow chart

Draw Flow Chart for Fake Coin Puzzle



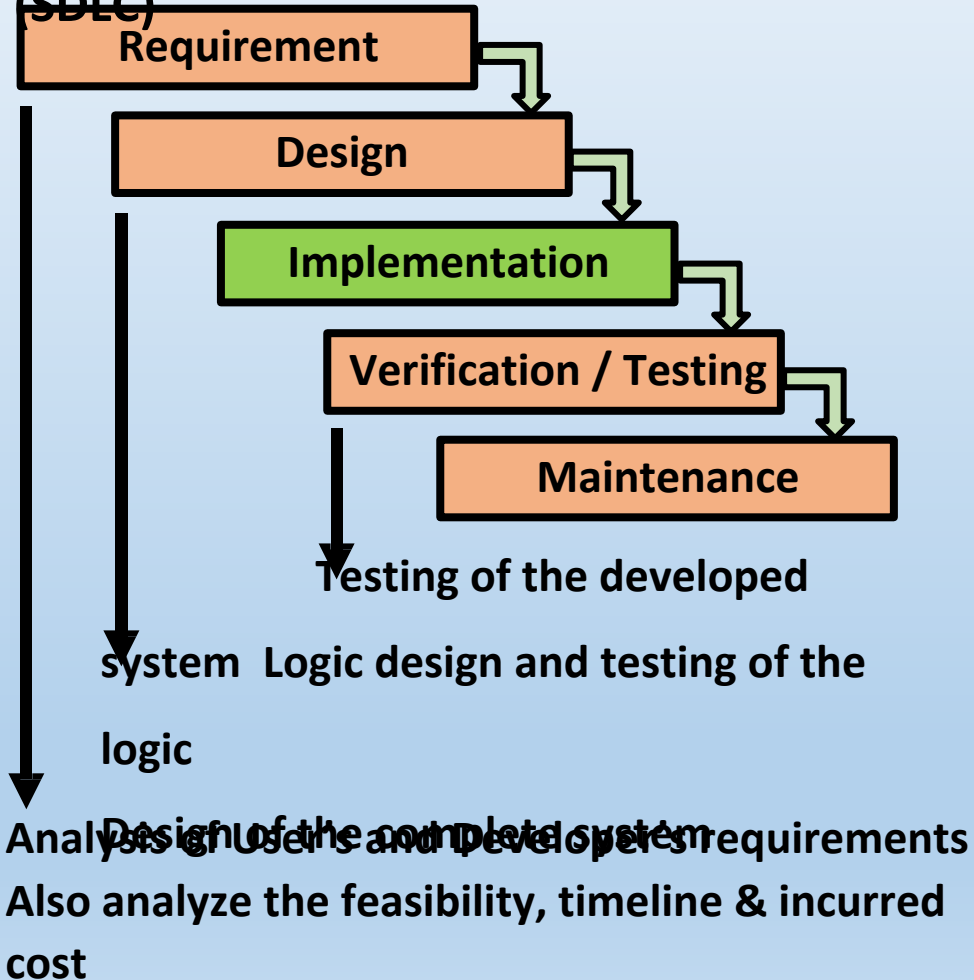
Weight of
Coins $W_1, W_2,$
... W_8



Implementation of Logic/Algorithm

Software Development Life Cycle

(SDLC)



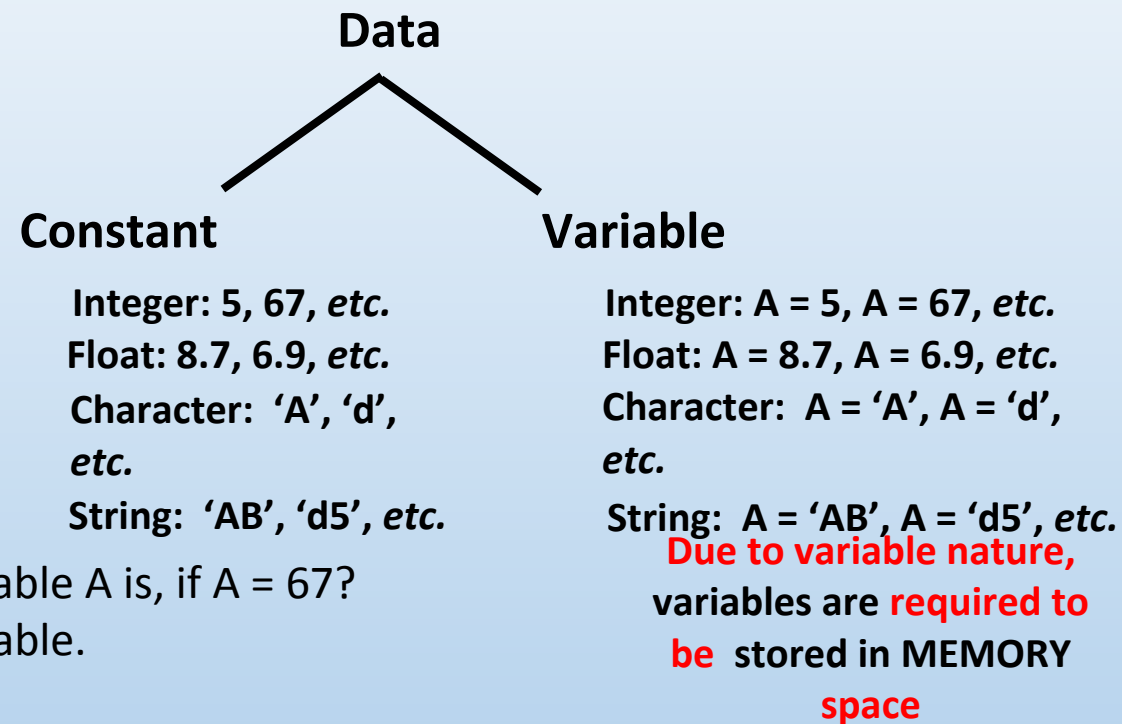
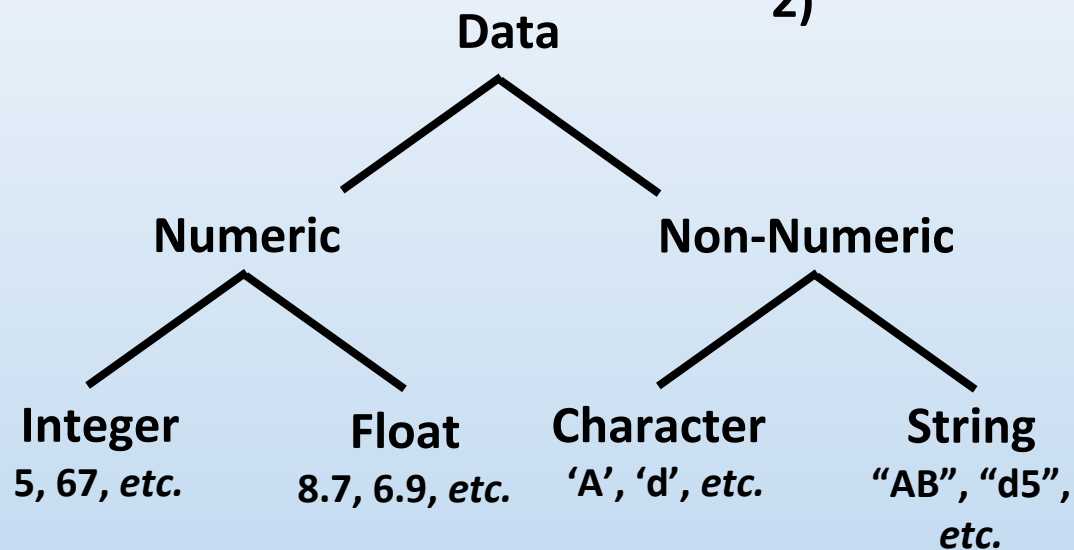
Logic/Algorithm is implemented using Programming Language
(C / C++ / Java / Python etc.)

Programming Language in SDF – I : C

Data Types
Input / Output
Operators
Control Flow
Array
Functions
Pointers
Structures/Union
Files

Data Types in C – A Brief Introduction

(We will discuss it in detail in Module 2)



Q1. Can you tell, what type of constant 5 is and what type of variable A is, if A = 67?

Ans. Yes. 5 is integer type constant and A is integer type variable.

Q2. Can you tell, what type of variable A is?

Ans. No. We do not know what type of data A is/will be holding/storing

Q3. How to know, what type of variable A is?

Ans. We need to declare the Data Type for A.

In C programming, 3 basic data types are often used: int (for Integer), float (for Float), and char (for Character)

e.g. int A □ A is integer type variable

In C programming, data type (int/float/char) of a variable must be declared before storing any value in it

Input / Output in C – A Brief Introduction

Input

scanf():

- (1) Reads the data for variables (V1, V2, etc.) in the C program.
- (2) Uses Format Specifier (FS) to tell the type of data for variables (V1, V2, etc.)
%d (int), %f (float), %c (char), etc.
- (3) Uses & to store the data at allotted memory space for the variables

Syntax: scanf("FS for V1 V2 etc.", &V1, &V2, etc.) e.g.: int V1, float V2, char V3

```
scanf("%d", &V1)
```

\t is used to insert tab and \n is used to insert new line while displaying using printf()

```
scanf("%d %d %c", &V1, &V2, &V3)
```

```
printf("JIIT Sec 62, Noida") □ JIIT Sec 62, Noida
```

```
printf("JIIT Sec 62, \n Noida") □ JIIT Sec 62,
```

Noida

Output

printf():

- (1) Display the data (variables / String constant) of a C program on monitor/screen.
- (2) Uses Format Specifier to tell the type of variable to display

Syntax printf("FS for V1 V2 etc.", V1, V2, (char), etc.

:

etc.) printf("String Constant")

printf("String Constant and FS for V1 V2 etc.", V1, V2, etc.)

e.g.: int V1, float V2, char V3

```
int V1, V2, V3; V3 = 'A'
```

□ 62

```
printf("%d %f %c", V1, V2, V3)
```

□ 62 9.5 A

```
printf("JIIT Sec 62, Noida")
```

□ JIIT Sec 62, Noida

```
printf("JIIT Sec %d, Noida",
```

□ JIIT Sec 62, Noida

```
V1)
```

```
printf()
```

/

```
whereas, printf("JIIT Sec 62, \t Noida") □ JIIT Sec 62,
```

Noida

Exercises

- Q1.** Constrained with, no accessibility to watches, draw the flow chart to find out the minimum count of races required to find three fastest horses, if count of horses is 25 and at a time only 5 horses can participate in a race.
- Q2.** Covering all the requirements of the Snake & Ladder game, draw the flow chart.
- Q3.** Draw the flow chart to verify whether the user inputted number is a prime number or not.

End of
Lecture 4